

8th EXERCISE: GENERATING A MATHEMATICAL SERIES

WHAT'S THE PROBLEM?

In this exercise, we need to find the value at a specific position in a special number sequence. The sequence follows these simple rules:

- *The first number (position 0) is 3*
- *To get the next number, we multiply the current number by 2 and add 1*

Our goal is to write a program that can tell us the number at any position in this sequence without using too much computer memory.

LET'S LOOK AT THE FIRST FEW NUMBERS

Let's write out the first few numbers in the sequence to see the pattern:

Position 0: 3 (this is our starting number)

Position 1: $3 \times 2 + 1 = 7$

Position 2: $7 \times 2 + 1 = 15$

Position 3: $15 \times 2 + 1 = 31$

Position 4: $31 \times 2 + 1 = 63$

Position 5: $63 \times 2 + 1 = 127$

Can you see the pattern? Each number is double the previous number, plus 1.

A REAL-WORLD ANALOGY

Imagine you have a special savings account. On day zero, you have £3. Each day, the bank doubles your money and adds £1 as a bonus. This exercise asks: how much money will you have after n days?

THE SMART WAY TO SOLVE IT

There are two ways we could solve this problem:

1. **The Obvious Way:** Start with 3, then apply the rule over and over until we reach the position we want
2. **The Clever Way:** Use a mathematical formula to calculate the answer directly

Let's start with the first approach because it's easier to understand.

SOLVING IT STEP-BY-STEP

To find the value at position n, we:

1. Start with the value 3
2. Apply the rule (multiply by 2 and add 1) n times
3. Return the final value

WHAT THIS LOOKS LIKE IN PYTHON CODE

```
def find_sequence_value(n):
    value = 3 # Start with the first number (position 0)

    for i in range(n):
        value = value * 2 + 1 # Apply our rule

    return value

print(find_sequence_value(5)) # Output: 127
```

This code is beautifully simple! We start with `value = 3`, then apply our rule `n` times in a loop. The variable `value` always holds the current number in the sequence.

LET'S SEE IT IN C

```
#include <stdio.h>

int find_sequence_value(int n) {
    int value = 3; // Start with the first number (position 0)

    for (int i = 0; i < n; i++) {
        value = value * 2 + 1; // Apply our rule
    }

    return value;
}

int main() {
    printf("%d\n", find_sequence_value(5)); // Output: 127
    return 0;
}
```

BASH VERSION (for shell script enthusiasts)

```
find_sequence_value() {
    local n=$1
    local value=3 # Start with the first number (position 0)

    for ((i=0; i<n; i++)); do
        value=$((value * 2 + 1)) # Apply our rule
    done

    echo "$value"
}

find_sequence_value 5 # Output: 127
```

LET'S TRACE THROUGH AN EXAMPLE

Let's follow the algorithm step by step for `n=5`:

1. Start with `value = 3`
2. Loop 5 times:
 - Iteration 1: `value = 3 * 2 + 1 = 7`
 - Iteration 2: `value = 7 * 2 + 1 = 15`
 - Iteration 3: `value = 15 * 2 + 1 = 31`
 - Iteration 4: `value = 31 * 2 + 1 = 63`
 - Iteration 5: `value = 63 * 2 + 1 = 127`
3. Return `value = 127`

A VISUAL STEP-BY-STEP

Here's a table showing how the value changes with each step:

Step	Calculation	New Value
------	-------------	-----------

Step	Calculation	New Value
Start	-	3
1	$3 \times 2 + 1$	7
2	$7 \times 2 + 1$	15
3	$15 \times 2 + 1$	31
4	$31 \times 2 + 1$	63
5	$63 \times 2 + 1$	127

THE CLEVER MATHEMATICAL SOLUTION

If we look closely at the numbers, we can spot a pattern:

Position 0: $3 = 2^2 - 1$

Position 1: $7 = 2^3 - 1$

Position 2: $15 = 2^4 - 1$

Position 3: $31 = 2^5 - 1$

Position 4: $63 = 2^6 - 1$

Position 5: $127 = 2^7 - 1$

Do you see it? Each value seems to follow the formula: $2^{n+2} - 1$

Let's check:

For position 5, we'd calculate $2^{5+2} - 1 = 2^7 - 1 = 128 - 1 = 127$.

It works!

We can implement this direct formula:

```
def find_sequence_value_direct(n):
    return (2 ** (n + 2)) - 1

print(find_sequence_value_direct(5)) # Output: 127
```

This solution is even faster because it doesn't need a loop!

WHY THIS FORMULA WORKS

Let's prove why this formula works mathematically. Our recurrence relation is:

$$a_0 = 3$$

$$a_n = a_{n-1} \times 2 + 1$$

If we assume that the formula $a_n = 2^{n+2} - 1$ is correct:

For a_0 :

$$a_0 = 2^{0+2} - 1 = 2^2 - 1 = 4 - 1 = 3 \checkmark$$

For the recursive step:

$$a_n = a_{n-1} \times 2 + 1$$

$$a_n = (2^{n+1} - 1) \times 2 + 1$$

$$a_n = 2^{n+2} - 2 + 1$$

$$a_n = 2^{n+2} - 1 \quad \checkmark$$

So the formula works! This is what mathematicians call a "*closed-form expression*" - a direct formula without needing to calculate previous terms.

COOL PATTERNS IN THIS SEQUENCE

This sequence has some interesting properties:

1. **Binary Pattern:** If you write each number in binary (base 2), you get a neat pattern:

- $3 = 11$ (in binary)
- $7 = 111$
- $15 = 1111$
- $31 = 11111$
- $63 = 111111$
- $127 = 1111111$

Each number is just a string of $(n+2)$ ones!

2. **Doubling Plus One:** Each number is slightly more than double the previous number.

3. **Special Numbers:** These are called "*Mersenne numbers*" in mathematics (numbers of the form $2^n - 1$).

HOW EFFICIENT IS OUR SOLUTION?

Let's look at both approaches:

Loop Approach:

- **Time Efficiency:** We need n steps to calculate the n th value ($O(n)$ in computer science terms)
- **Memory Efficiency:** We only need to store a single value regardless of how big n is ($O(1)$ space)

Direct Formula Approach:

- **Time Efficiency:** We calculate the value directly ($O(1)$ time, though calculating powers can take longer for very large numbers)
- **Memory Efficiency:** Also $O(1)$ space (we only need a few variables)

Both approaches are efficient in terms of memory, which was our goal!

REAL-WORLD APPLICATIONS

This type of sequence and the techniques we used have applications in:

1. **Computer Science:** Similar patterns appear when analyzing many algorithms
2. **Finance:** Modelling compound interest with regular additions
3. **Number Theory:** The Mersenne numbers we discovered have connections to prime numbers

GOING FURTHER: CHALLENGES YOU COULD TRY

Once you understand the basic problem, you might try:

1. **Very Large Numbers:** Can you calculate values for large positions (like $n=1000$) without causing overflow?
2. **Reverse Problem:** If I tell you a number from the sequence (like 8191), can you tell me its position?
3. **Different Rules:** What if we changed the rule to "multiply by 3 and add 2"? Can you find the pattern?

WHAT WE'VE LEARNED

This exercise teaches us:

1. **Recognizing Patterns:** Looking for patterns in sequences
2. **Iterative Calculation:** Building each value step by step
3. **Mathematical Insight:** Finding clever formulas to solve problems directly
4. **Efficiency Awareness:** Understanding the importance of memory-efficient solutions

The beauty of this problem is that it can be solved both through straightforward iteration and through clever mathematical insight - two important approaches in programming and problem-solving.